# LZW Encoding of a 6×6 Data Matrix

## 1. Problem Statement:

The goal of this assignment is to encode a 6×6 matrix of integer data using the LZW (Lempel–Ziv–Welch) compression algorithm. LZW is a lossless data compression technique that builds a dictionary of sequences dynamically and replaces repeated patterns with shorter codes.

## 2. Input Data:

The input is a 6×6 matrix of integers:

$$\begin{bmatrix} 36 & 36 & 36 & 126 & 126 & 126 \\ 36 & 36 & 36 & 126 & 126 & 126 \\ 36 & 36 & 36 & 126 & 126 & 126 \\ 36 & 36 & 36 & 126 & 126 & 126 \\ 36 & 36 & 36 & 126 & 126 & 126 \\ 36 & 36 & 36 & 126 & 126 & 126 \end{bmatrix}$$

Each value represents a symbol to be encoded.

## 3. Steps of LZW Encoding:

### Step 1: Flatten the Data

LZW encoding requires a 1D sequence instead of a 2D matrix.

```
data_lst = []
for i in range(6):
    for j in range(6):
        data_lst.append(int(data[i][j]))
```

After flattening, the sequence becomes: [39, 39, 39, 126, 126, 126 ……..]

Total elements = **36**

### Step 2: Initialize Dictionary

Initially, the dictionary contains all possible single-byte symbols (0–255):

```
dictionary = {tuple([i]): i for i in range(256)}
dict_size = 256
```

So:

- (39) → 39
- (126) → 126
- etc.

## Step 3: Initialize Variables

```
w = ()
result = []
```

- w: current sequence
- result: compressed output codes

## Step 4: Encoding Process

For each symbol in the sequence:

```
for i in data_lst:
    wk = w + (i,)
    if wk in dictionary:
        w = wk
    else:
        result.append(dictionary[w])
        dictionary[wk] = dict_size
        dict_size += 1
        w = (i,)
```

## Explanation:

- Combination of current sequence `w` with next symbol: `wk`
- If `wk` exists in dictionary then extend sequence
- Otherwise:
    - Output code for `w`
    - Add `wk` to dictionary
    - Reset `w`

## Step 5: Final Output

```
if w:
    result.append(dictionary[w])
```

This ensures the last sequence is also encoded.

## 4. Working Example:

| Step | Input | w (before) | wk | Action | Output | New Dictionary Entry |
|---|---|---|---|---|---|---|
| 1 | 39 | () | (39) | Exists → w=wk | - | - |
| 2 | 39 | (39) | (39,39) | Not found | 39 | 256 → (39,39) |
| 3 | 39 | (39) | (39,39) | Exists → w=wk | - | - |
| 4 | 126 | (39,39) | (39,39,126) | Not found | 256 | 257 → (39,39,126) |
| 5 | 126 | (126) | (126,126) | Not found | 126 | 258 → (126,126) |
| 6 | 126 | (126) | (126,126) | Exists → w=wk | - | - |
| 7 | 39 | (126,126) | (126,126,39) | Not found | 258 | 259 → (126,126,39) |
| 8 | 39 | (39) | (39,39) | Exists → w=wk | - | - |
| 9 | 39 | (39,39) | (39,39,39) | Not found | 256 | 260 → (39,39,39) |
| 10 | 126 | (39) | (39,126) | Not found | 39 | 261 → (39,126) |
| 11 | 126 | (126) | (126,126) | Exists → w=wk | - | - |
| 12 | 126 | (126,126) | (126,126,126) | Not found | 258 | 262 → (126,126,126) |
| 13 | 39 | (126) | (126,39) | Not found | 126 | 263 → (126,39) |
| 14 | 39 | (39) | (39,39) | Exists → w=wk | - | - |
| 15 | 39 | (39,39) | (39,39,39) | Exists → w=wk | - | - |
| 16 | 126 | (39,39,39) | (39,39,39,126) | Not found | 260 | 264 → (39,39,39,126) |
| 17 | 126 | (126) | (126,126) | Exists → w=wk | - | - |
| 18 | 126 | (126,126) | (126,126,126) | Exists → w=wk | - | - |
| 19 | 39 | (126,126,126) | (126,126,126,39) | Not found | 262 | 265 → (126,126,126,39) |
| 20 | 39 | (39) | (39,39) | Exists → w=wk | - | - |
| 21 | 39 | (39,39) | (39,39,39) | Exists → w=wk | - | - |
| 22 | 126 | (39,39,39) | (39,39,39,126) | Exists → w=wk | - | - |
| 23 | 126 | (39,39,39,126) | (39,39,39,126,126) | Not found | 264 | 266 → (39,39,39,126,126) |
| 24 | 126 | (126) | (126,126) | Exists → | - | - |

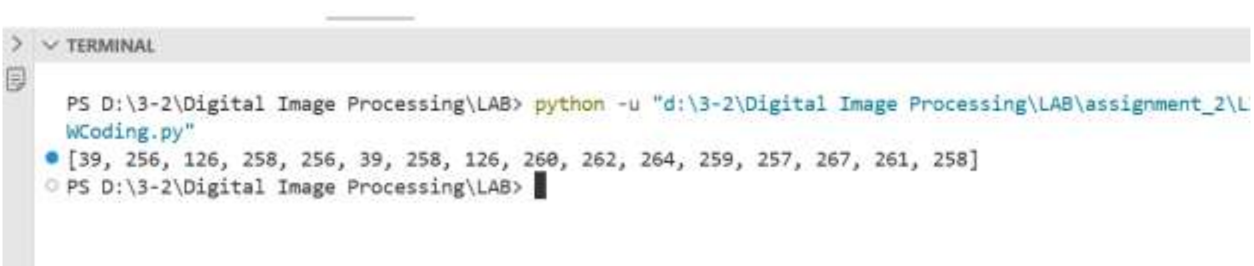
| | | | | | | |
|---|---|---|---|---|---|---|
| | | | | w=wk | | |
| 25 | 39 | (126,126) | (126,126,39) | Exists → w=wk | - | - |
| 26 | 39 | (126,126,39) | (126,126,39,39) | Not found | 259 | 267 → (126,126,39,39) |
| 27 | 39 | (39) | (39,39) | Exists → w=wk | - | - |
| 28 | 126 | (39,39) | (39,39,126) | Exists → w=wk | - | - |
| 29 | 126 | (39,39,126) | (39,39,126,126) | Not found | 257 | 268 → (39,39,126,126) |
| 30 | 126 | (126) | (126,126) | Exists → w=wk | - | - |
| 31 | 39 | (126,126) | (126,126,39) | Exists → w=wk | - | - |
| 32 | 39 | (126,126,39) | (126,126,39,39) | Exists → w=wk | - | - |
| 33 | 39 | (126,126,39,39) | (126,126,39,39,39) | Not found | 267 | 269 → (126,126,39,39,39) |
| 34 | 126 | (39) | (39,126) | Exists → w=wk | - | - |
| 35 | 126 | (39,126) | (39,126,126) | Not found | 261 | 270 → (39,126,126) |
| 36 | 126 | (126) | (126,126) | Exists → w=wk | - | - |

## 5. Encoded Output:

After running the algorithm:

```
print(result)
```

We will get a list of integer codes representing the compressed data.




## 6. Source Code:

```
import numpy as np
```

```python
data = np.array([
    [39, 39, 39, 126, 126, 126],
    [39, 39, 39, 126, 126, 126],
    [39, 39, 39, 126, 126, 126],
    [39, 39, 39, 126, 126, 126],
    [39, 39, 39, 126, 126, 126],
    [39, 39, 39, 126, 126, 126],
])

data_lst = []
for i in range(6):
    for j in range(6):
        data_lst.append(int(data[i][j]))

dictionary = {tuple([i]): i for i in range(256)}
dict_size = 256

w = ()
result = []

for i in data_lst:
    wk = w + (i,)
    if wk in dictionary:
        w = wk
    else:
        result.append(dictionary[w])
        dictionary[wk] = dict_size
        dict_size += 1
        w = (i,)

if w:
    result.append(dictionary[w])

print(result)
```